

Tokens to Translations

Summer of Code 2026

Midterm Report

Name: Samyajyoti Biswas
Department: Computer Science and Engineering
Institute: IIT Bombay
Mentors: Tushar Yadav
Keshav Singhal
Date: June 21, 2026

Contents

1	Introduction	2
2	Week 1: Neural Networks and Character-Level Language Modeling	2
2.1	Automatic Differentiation and Neural Networks	2
2.2	Character-Level Language Modeling	2
2.3	Neural Bigram Model	3
3	Week 2: Multi-Layer Perceptron Language Models	3
3.1	Context-Based Prediction	3
3.2	Embedding Layers	4
3.3	Multi-Layer Perceptron Language Model	4
3.4	Batch Normalization	4
3.5	Kaiming Initialization	5
3.6	Implementing Layers from Scratch	5
4	Week 3: Deep Network Optimization and Architectural Improvements	5
4.1	WaveNet-Inspired Architectures	5
5	Key Outcomes	6
6	Conclusion	6
	References	7

1 Introduction

The objective of this project is to understand how neural networks learn representations of language and how these representations can eventually be used to perform translation tasks. The first phase of the project therefore focuses on language modeling, optimization, and neural network fundamentals. This report summarizes the work completed during the first three weeks of the project. Rather than treating neural networks and translation systems as black boxes, the project focuses on building the fundamental components from scratch, beginning with automatic differentiation and neural networks, progressing through character-level language models and multi-layer perceptrons, and finally exploring modern architectural improvements such as Batch Normalization, Kaiming Initialization, and hierarchical context aggregation inspired by WaveNet.

2 Week 1: Neural Networks and Character-Level Language Modeling

2.1 Automatic Differentiation and Neural Networks

The project began with an implementation of automatic differentiation inspired by the Micrograd framework.

A custom `Value` class was implemented to represent scalar values and maintain computational graphs. Arithmetic operations were overloaded to construct computation graphs automatically, allowing gradients to be propagated backward through the graph.

The following concepts were explored:

- Numerical differentiation
- Computational graphs
- Chain rule
- Backpropagation
- Gradient descent
- Multi-Layer Perceptrons (MLPs)

A neural network consisting of neurons, layers, and a complete MLP was implemented from scratch. Training was performed using gradient descent and manually computed gradients obtained through automatic differentiation.

This phase provided a detailed understanding of how modern deep learning frameworks internally compute gradients and update parameters.

2.2 Character-Level Language Modeling

The next stage involved implementing a character-level language model using a dataset of names.

Initially, a bigram language model was constructed by counting occurrences of consecutive characters. These counts were converted into probability distributions and used to generate new names through probabilistic sampling.

The model learned distributions of the form

$$P(c_t \mid c_{t-1})$$

where the next character depends only on the previous character.

Key concepts explored:

- Character tokenization
- Bigram statistics
- Probability matrices
- Sampling from learned distributions
- Negative Log-Likelihood (NLL)
- Model smoothing

2.3 Neural Bigram Model

The statistical bigram model was then reformulated as a neural network.

Characters were represented using one-hot encodings and passed through a trainable weight matrix. The resulting logits were converted into probabilities using a softmax layer and optimized using cross-entropy loss.

This experiment demonstrated how neural networks can learn probability distributions directly from data rather than relying on explicit counting procedures.

3 Week 2: Multi-Layer Perceptron Language Models

3.1 Context-Based Prediction

The second phase of the project focused on extending the bigram model to incorporate larger contexts.

Instead of predicting the next character from only the immediately preceding character, the model was trained using multiple previous characters as context.

The dataset was divided into:

- Training Set
- Development Set
- Test Set

A context window was used to predict the next character in a sequence.

3.2 Embedding Layers

One-hot encoded inputs become increasingly inefficient as vocabulary size grows.

To address this limitation, embedding vectors were introduced.

Each character was mapped to a trainable dense vector representation, allowing the model to learn relationships between characters in a continuous vector space.

This stage introduced:

- Embedding lookups
- Dense representations
- Context concatenation
- Hidden layer representations

3.3 Multi-Layer Perceptron Language Model

A Multi-Layer Perceptron was trained to predict the next character given a context window.

The architecture consisted of:

- Embedding Layer
- Hidden Layer with non-linearity
- Output Layer producing logits

Several hyperparameters were investigated, including:

- Context size
- Hidden layer width
- Learning rate
- Embedding dimension

The best-performing configuration achieved a validation loss of approximately 2.14.

This phase was inspired by the ideas presented in Bengio et al. (2003), *A Neural Probabilistic Language Model*, and demonstrated how distributed representations can be used to capture structure in language.

3.4 Batch Normalization

As network depth increased, training stability became increasingly important.

Batch Normalization was implemented and studied in detail.

Experiments focused on:

- Activation distributions

- Gradient flow
- Training stability
- Optimization behavior

The effect of BatchNorm on optimization was analyzed by monitoring activation statistics throughout the network.

3.5 Kaiming Initialization

Weight initialization strategies were investigated to improve signal propagation through deep networks.

In particular, Kaiming Initialization was implemented and compared against naive random initialization.

Experiments demonstrated that initialization has a substantial effect on:

- Activation magnitudes
- Gradient propagation
- Training convergence

3.6 Implementing Layers from Scratch

To better understand modern deep learning frameworks, custom implementations of neural network layers were developed.

These included:

- Linear Layers
- Batch Normalization Layers
- Activation Functions

The objective was to create abstractions similar to those provided by the PyTorch API while maintaining full control over the underlying computations.

4 Week 3: Deep Network Optimization and Architectural Improvements

4.1 WaveNet-Inspired Architectures

The final component completed during this period involved exploring hierarchical context aggregation strategies inspired by WaveNet.

A key observation was that previous MLP models compressed the entire context window in a single step. To address this limitation, a hierarchical architecture was introduced using custom layers such as:

- FlattenConsecutive
- Hierarchical feature aggregation
- Progressive context compression

Rather than combining all contextual information simultaneously, information was compressed gradually across multiple stages.

This allowed larger context windows to be processed more efficiently while maintaining parameter efficiency.

5 Key Outcomes

By the end of the first three weeks, the following components had been successfully implemented:

- Automatic differentiation engine
- Backpropagation
- Multi-Layer Perceptrons
- Statistical bigram language models
- Neural bigram language models
- Embedding layers
- Context-based language models
- Batch Normalization
- Kaiming Initialization
- Custom neural network layers
- WaveNet-inspired hierarchical architectures

These implementations established the mathematical and computational foundations required for studying more advanced sequence models and eventually modern machine translation architectures.

6 Conclusion

The first three weeks of the project focused on developing a strong foundation in neural networks and language modeling through direct implementation. Beginning with automatic differentiation and progressing through neural language models, optimization techniques, and hierarchical architectures, the project has provided practical insight into how modern language models are constructed and trained. These foundations will serve as the basis for studying attention-based models and machine translation systems in the subsequent phases of the project.

References

1. Karpathy, A. *Neural Networks: Zero to Hero*.
2. Bengio, Y. et al. *A Neural Probabilistic Language Model*, 2003.
3. WaveNet: A Generative Model for Raw Audio.
4. PyTorch Documentation.